LangChain Section 10

Callbacks, Streaming & Tracing

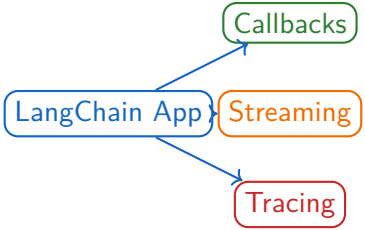
LangChain v1.2.x (March 2026)

Observability, performance monitoring, and real-time event streaming

Mental Model: Observability as First-Class

Modern LLM apps need visibility into every step

- **What's happening?** Trace execution flow
- **When is it happening?** Track latency and timing
- **What's it costing?** Monitor token usage
- **Is it working?** Debug failures in production



[Docs: Observability in LangChain]

Why Observability Matters

Development:

- Debug complex chains
- Understand token flow
- Identify bottlenecks
- Test edge cases

Production:

- Monitor cost/usage
- Catch failures early
- Improve performance
- Audit decisions

Key Insight

Without observability, LLM systems become black boxes

LangChain provides three complementary tools:

1. **Callbacks** — react to events in real-time
2. **Streaming** — get results token-by-token
3. **Tracing** — capture complete execution history

The Callback System: BaseCallbackHandler

Core Idea

Callbacks let you hook into LangChain events and react to them

```python
from langchain_core.callbacks import BaseCallbackHandler

class MyCallback(BaseCallbackHandler):
    """Custom handler for LangChain events."""

    def on_llm_start(self, serialized, inputs, **kwargs):
        print(f"LLM starting with: {inputs}")

    def on_llm_end(self, response, **kwargs):
        print(f"LLM finished: {response}")
```

Key methods exist for:

- LLM lifecycle: `on_llm_start`, `on_llm_end`, `on_llm_error`
- Tool usage: `on_tool_start`, `on_tool_end`
- Chain execution: `on_chain_start`, `on_chain_end`
- Token streaming: `on_llm_new_token`

[Docs: BaseCallbackHandler API]

Built-in Callback Events

Event	Triggered When
`on_llm_start`	LLM call begins
`on_llm_end`	LLM call completes
`on_llm_new_token`	Token streamed from LLM
`on_tool_start`	Tool invocation begins
`on_tool_end`	Tool invocation completes
`on_chain_start`	Chain/runnable starts
`on_chain_end`	Chain/runnable finishes
`on_retriever_start`	Retriever search begins
`on_retriever_end`	Retriever search completes
`on_text`	Text output generated

Pro Tip

Not all handlers need to implement all methods — implement only what you need

[Docs: Callback Events Reference]

Creating Custom Callback Handlers

```python
from langchain_core.callbacks import BaseCallbackHandler
import json

class LoggingCallback(BaseCallbackHandler):
    def on_llm_start(self, serialized, inputs, **kwargs):
        print(f"Model: {serialized.get('name')}")
        print(f"Prompt: {inputs.get('prompts', [])}")

    def on_llm_end(self, response, **kwargs):
        msg = response.generations[0][0].text
        tokens = response.llm_output.get('usage', {})
        print(f"Output: {msg}")
        print(f"Tokens: {json.dumps(tokens)}")

    def on_llm_error(self, error, **kwargs):
        print(f"ERROR: {error}")
```

Async Callbacks

For async chains, implement `on_llm_start` (async version exists too)

[Docs: Custom Callbacks]

Streaming Token-by-Token: stream() & astream()

```python
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", streaming=True)

# Synchronous streaming
for chunk in llm.stream("Explain quantum computing"):
    print(chunk.content, end="", flush=True)

# Asynchronous streaming
async def stream_response():
    async for chunk in llm.astream("Explain quantum"):
        print(chunk.content, end="", flush=True)

import asyncio
asyncio.run(stream_response())
```

When to Use

Use `stream()` for real-time user feedback, `astream()` for web apps

[Docs: Streaming Reference]

astream_events() — Universal Streaming Interface

```python
from langchain_openai import ChatOpenAI
from langchain_core.callbacks import AsyncIteratorCallbackHandler

llm = ChatOpenAI(model="gpt-4o")

# Stream ALL events from a runnable
async def main():
    async for event in llm.astream_events(
        {"messages": [("human", "Hello")]},
        version="v2"
    ):
        print(f"{event['event']}: {event}")

# Events include:
# - on_llm_start, on_llm_stream, on_llm_end
# - on_chain_start, on_chain_stream, on_chain_end
```

Advantage

Single interface handles all runnable types uniformly

[Docs: astream_events Documentation]

Event Types and How to Filter Them

```python
async def process_events():
    async for event in llm.astream_events(prompt):
        event_type = event["event"]

        # Filter by type
        if event_type == "on_llm_stream":
            token = event["data"]["chunk"].content
            print(f"Token: {token}")

        elif event_type == "on_chain_end":
            result = event["data"]["output"]
            print(f"Final: {result}")

        # Ignore internal events
        if not event_type.startswith("on_"):
            continue
```

Common event structures:

- `event` — type name (`"on_llm_start"`, etc.)
- `data` — event data (chunks, outputs, errors)
- `run_id` — trace identifier for correlation

[Docs: Event Structure]

Streaming in Web Apps (FastAPI/SSE)

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from langchain_openai import ChatOpenAI

app = FastAPI()
llm = ChatOpenAI(model="gpt-4o")

@app.post("/chat/stream")
async def chat_stream(message: str):
    async def generate():
        async for chunk in llm.astream(message):
            yield f"data: {chunk.content}\n\n"

    return StreamingResponse(
        generate(),
        media_type="text/event-stream"
    )
```

Client side (JavaScript):

```javascript
const es = new EventSource("/chat/stream?message=...");
es.onmessage = (e) => console.log(e.data);
es.onerror = () => es.close();
```

[Docs: FastAPI + Streaming Example]

LangSmith Tracing: Automatic Capture

LangSmith is LangChain's observability platform

What it captures:

- Full execution traces
- Token usage (prompt + completion)
- Latency per step
- Inputs and outputs
- Errors and exceptions
- Custom metadata

Features:

- Web dashboard
- Performance analytics
- A/B testing
- Dataset curation
- Debugging tools

Key Point

Tracing is automatic — zero code changes needed

[Docs: LangSmith Documentation]

Setting Up LangSmith

1. **Get API key:** Sign up at `smith.langchain.com`
2. **Set environment variable:**

```
# In shell or .env file
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_API_KEY="ls_..."
export LANGCHAIN_PROJECT="my-project"
```

3. **Run your code** — traces appear automatically

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o")
response = llm.invoke("Hello!")
# ^ Automatically traced to LangSmith
```

Important

Never commit your API key! Use environment variables

[Docs: LangSmith Setup]

Reading and Interpreting Traces

LangSmith dashboard shows:

- **Trace graph** — hierarchical execution flow
- **Latency** — time per step (ms)
- **Token count** — input/output tokens
- **Cost** — estimated API cost
- **Inputs/Outputs** — full data logged
- **Errors** — exceptions with stack traces
- **Metadata** — custom tags/context
- **Feedback** — human ratings

How to Use

Use traces to identify slowdowns, understand token usage, and debug failures

[Docs: Reading Traces]

Adding Metadata and Tags to Traces

```python
from langchain_openai import ChatOpenAI
from langchain_core.callbacks import CallbackConfig
llm = ChatOpenAI(model="gpt-4o")
response = llm.invoke(
    "Hello",
    config={
        "run_name": "greeting",
        "tags": ["production", "v1.2"],
        "metadata": {
            "user_id": "user_123",
            "session": "abc-def",
            "experiment": "temperature_0.7"
        }
    }
)
```

Benefits:

- Filter traces in LangSmith by tags
- Correlate with external systems
- Track A/B experiments
- Organize by user, session, or version

[Docs: Metadata Configuration]

Custom Spans with @traceable Decorator

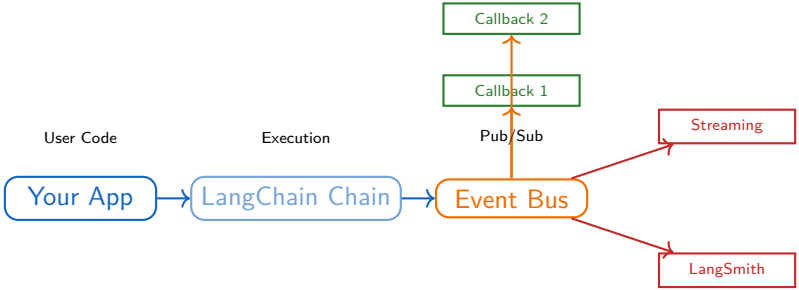
```python
from langsmith import traceable
from langchain_openai import ChatOpenAI

@traceable
def summarize(text: str) -> str:
    """Custom function traced as a span."""
    llm = ChatOpenAI(model="gpt-4o")
    response = llm.invoke(f"Summarize: {text}")
    return response.content

@traceable
async def process_documents(docs: list) -> list:
    """Async function with tracing."""
    results = []
    for doc in docs:
        summary = summarize(doc)  # Nested trace
        results.append(summary)
    return results

# Use normally - traces captured automatically
result = summarize("Long text here...")
```

Benefit

Nest custom functions and see full hierarchy in LangSmith

[Docs: @traceable Decorator]

Callback/Trace Data Flow Architecture



All execution flows through the event bus, enabling parallel observability

Performance Monitoring: Latency, Tokens, Cost

```python
from langchain_openai import ChatOpenAI
import time

class MetricsCallback:
    def on_llm_start(self, **kwargs):
        self.start = time.time()

    def on_llm_end(self, response, **kwargs):
        elapsed = time.time() - self.start
        tokens = response.llm_output.get("usage", {})

        print(f"Latency: {elapsed:.2f}s")
        print(f"Input tokens: {tokens.get('prompt_tokens')}")
        print(f"Output tokens: {tokens.get('completion_tokens')}")

        # Estimate cost (GPT-4o: $0.03/$0.06 per 1M tokens)
        total = tokens.get('prompt_tokens', 0) + \
                tokens.get('completion_tokens', 0)
        cost = total / 1_000_000 * 0.045
        print(f"Est. cost: ${cost:.4f}")
```

[Docs: Performance Monitoring]

Debugging with Verbose Mode and Trace Inspection

```python
from langchain_openai import ChatOpenAI
from langchain.callbacks import StdOutCallbackHandler

llm = ChatOpenAI(
    model="gpt-4o",
    verbose=True,  # Enable verbose logging
    callbacks=[StdOutCallbackHandler()]
)

response = llm.invoke("Explain neural networks")
# Outputs: Full trace to stdout
```

Verbose output includes:

- Prompt sent to LLM
- Model response
- Token counts
- Latency

Caution

Verbose mode can be noisy — use selectively in development

[Docs: Verbose Debugging]

Common Pitfalls and How to Avoid Them

Pitfall 1: Blocking Callbacks

Async chains with sync callbacks can cause deadlocks. Use `async def` in async contexts.

```
# WRONG: Sync callback in async chain
class BadCallback(BaseCallbackHandler):
    def on_llm_end(self, response):
        time.sleep(1)  # Blocks event loop!

# RIGHT: Async callback
class GoodCallback(BaseCallbackHandler):
    async def on_llm_end(self, response):
        await asyncio.sleep(1)  # Non-blocking
```

Other pitfalls:

- **Missing imports** — ensure callbacks in scope
- **Wrong event names** — check case and spelling
- **API key errors** — verify `LANGCHAIN_API_KEY` set
- **Too much logging** — sample events in production

[Docs: Debugging Callbacks]

Summary: Callbacks, Streaming & Tracing

Three Pillars of Observability:

Tool	Purpose	When
Callbacks	React to events in real-time	Dev & prod
Streaming	Get results token-by-token	Web UIs
Tracing	Capture full execution history	Analysis & debug

Quick Start:

1. **Add callbacks:** Extend `BaseCallbackHandler`
2. **Stream results:** Use `.stream()` or `.astream()`
3. **Enable tracing:** Set env vars (`LANGCHAIN_TRACING_V2`)
4. **Monitor:** Check LangSmith dashboard

[Docs: Section 10 Complete]

Quick Reference Cheat Sheet (Part 1)

Custom Callback:

```
from langchain_core.callbacks import BaseCallbackHandler

class MyCallback(BaseCallbackHandler):
    def on_llm_end(self, response, **kw):
        print(response)
```

Streaming:

```
for chunk in llm.stream(prompt):
    print(chunk.content)
```

Async Streaming:

```
async for chunk in llm.astream(prompt):
    print(chunk.content)
```

Quick Reference Cheat Sheet (Part 2)

Event Streaming:

```
async for event in llm.astream_events(prompt):
    if event["event"] == "on_llm_stream":
        print(event["data"]["chunk"].content)
```

LangSmith Setup:

```
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_API_KEY="ls_..."
# Then run code normally
```

Add Metadata:

```
llm.invoke(prompt, config={
    "tags": ["prod"],
    "metadata": {"user": "id"}
})
```

Self-Check: Do You Understand?

1. What are the three main observability tools in LangChain?
2. When would you use callbacks vs. streaming vs. tracing?
3. How do you create a custom callback handler?
4. What's the difference between `stream()` and `astream_events()`?
5. How do you set up LangSmith tracing? (3 steps)
6. What's a common pitfall when using callbacks in async chains?
7. How can you add custom metadata to traces?

Answers: See previous slides and documentation links

Key Takeaways

1. Observability First

Build tracing and monitoring into your LLM apps from day one

2. Three Complementary Tools

Callbacks for hooks, streaming for UX, tracing for analysis

3. LangSmith

Minimal setup for powerful insights — use it!

4. Production Readiness

Monitoring, debugging, and cost tracking are non-negotiable

Next Steps

Immediate:

1. Implement a custom callback in your current project
2. Enable streaming for user-facing features
3. Sign up for LangSmith and enable tracing

Deeper exploration:

- Build a performance dashboard using callback metrics
- Create custom spans for domain-specific logic
- Set up alerts based on trace metrics
- Use LangSmith datasets for evaluation

Resources:

- LangChain callbacks docs
- LangSmith platform guide
- Example projects in langchain-ai/langchain repo

Questions?

LangChain v1.2.x

Section 10: Callbacks, Streaming & Tracing

Covered:

- Callback system and custom handlers
- Real-time streaming and events
- LangSmith tracing and observability
- Performance monitoring and debugging